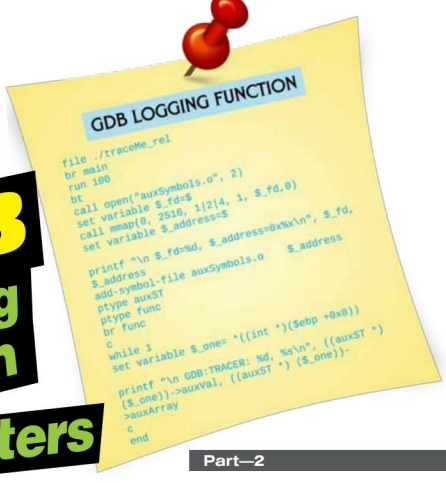

GDB Logging Function Parameters

Part—2

In this second part of the article, let us write a GDB script to log the parameters of a desired function, and also see how the application state can be modified by changing the desired parameters. This will help understand how to inject debug symbols in a GDB session, while debugging an inferior process that is built in release mode.

Basic concepts

In Part 1 of this series, we created an application, *traceMe_rel*, which can be found at *http://linuxforu.com/article_source_code/dec11/trace.tar.gz*. Since this application is in release mode, it does not have enough information about the type of arguments of function *func*, called in this application.

As a developer, you know the argument types of *func*, so you need to somehow cast it to the correct type (i.e., *ST*). This can be done by writing an *auxiliary* program in C (let us name it *auxSymbols.c*), the sole purpose of which is to inject debug symbols into GDB when debugging *traceMe_rel*. Let us compile *auxSymbols.c* in debug mode (passing *–g* to GCC) and load the symbols from this *auxiliary* module (it could be in object code form like *auxSymbols.o*, or a shared library like *libauxSym.so*). The benefits of this approach are that we do not change the original application binary at all, and that the only symbols the auxiliary program needs to define are the ones we need to target, e.g., *ST*. We will discuss the details below, but first, let's review our sample program and write our *auxiliary* program.

Note: The details discussed here have been tested on Fedora 15 running on an x86 platform, with GCC version 4.6.0, configured to produce 32-bit target. The GDB version is 7.2.90.20110429-36.fc15. The sample binaries produced in the article are all 32-bit.

Let us look at the same program (*traceMe.c*) used in Part 1, as an example. There is a UDT (user defined type) *ST* with two fields: *val* of type *int* and *arrayArg* of type *char[15]*. The program expects a non-zero positive number as a command-line parameter. The *main* function assigns the number to *st.val* and a string representation of it to *st.arrayArg*, where *st* is a variable of type *ST*. Then *main* calls *func*, which takes the address of *st* as a parameter. *func* is a recursive function. A recursive call to *func* is made after reducing *st.val* by a random number. The boundary condition for *func* occurs if *st.val* reduces to 0. Here is the output of running this program with an argument of 100:

```
[raman@Chalotra Part2]$ ./traceMe_rel 100
 Entered [1    ] with st->val=100            , st-
```